Implementing GraphRAG with PageRank for Efficient Multi-hop Query

Silviana
College of Computing and Data Science
Nanyang Technological University
Singapore

Abstract

This report documents a prototype web application that explores Graph Retrieval-Augmented Generation (GraphRAG) with Personalized PageRank as a graph-based alternative to conventional vector-similarity retrieval. The system accepts arbitrary URLs and text as input, extracts a knowledge graph using a large language model (Google Gemini), and answers multi-hop queries by computing personalized pagerank scores of the nodes in the graph with power method. The implementation is written in Rust, containerised with a distroless Docker image, and deployed to Azure Container Apps. This report is primarily a reflection on the learning process, the conceptual shifts, engineering challenges, and the many places where the current prototype falls short. The goal was not to build something production-ready, but to develop hands-on intuition to understand the process of building graph-based retrieval system.

1 Introduction

Retrieval-Augmented Generation (RAG) has become a common pattern for grounding large language model (LLM) responses with external knowledge. The basic idea is to retrieve relevant documents at query time and inject them into the prompt so the model has access to information beyond its training data. In practice, most RAG implementations rely on dense vector embeddings: documents are chunked, encoded into high-dimensional vectors, stored in a vector database, and retrieved by nearest-neighbour search at query time.

This works surprisingly well for many use cases, but it has a structural limitation. Vector similarity is fundamentally a measure of semantic resemblance between a query and a document chunk. It does not naturally capture multi-hop relationships, where the answer to a question requires traversing a chain of connected entities. For example, the question "What discoveries by Marie Curie led to later advances in medical imaging?" requires connecting Curie's work on radioactivity, to radium, to its therapeutic applications, to modern radiotherapy. This path spans multiple intermediate concepts. A vector search over document chunks may retrieve relevant paragraphs, but it does not have the capability to reason about the structure of the relationships between those concepts.

1.1 Traditional RAG Systems

A conventional RAG pipeline typically consists of two components: ingestion and query [4]. In the ingestion phase, documents are split into semantically coherent chunks, each chunk is encoded into a vector using an embedding model, and those vectors are indexed in a vector database. At query time, the query is embedded using the same model, and the top-k most similar chunks are retrieved by cosine similarity or inner product search. The retrieved chunks are appended to the prompt and the LLM generates a response conditioned on that context.

The key strengths of this approach are in its simplicity and scalability. Embedding and similarity search are well-documented, and the pipeline can be built using mature tooling. The weaknesses become apparent with queries that require reasoning across multiple documents or queries that depend on understanding how concepts relate to one another. For example, a chunk about "radiotherapy" may individually score poorly against the query about Curie's contributions to medical imaging.

1.2 GraphRAG

GraphRAG replaces or supplements the flat chunk index with a structured knowledge graph. Entities are extracted from source documents and represented as nodes. Relationships between entities are represented as directed edges. Retrieval then becomes a graph traversal problem rather than a nearest-neighbour search problem.

Relevant information in a multi-hop query can be located by starting from seed entities extracted from the query, then walking through the graph's edge to reveal nodes that are transitively connected to those seed entities. This is analogous to how PageRank works for web pages, but personalised to a specific query context.

2 Literature Review

2.1 Microsoft GraphRAG

Microsoft Research published a GraphRAG framework [2] that introduced the idea of using community detection over a knowledge graph to support queries that require traversing information across an entire document corpus rather than retrieving specific facts. Their pipeline involves LLM-driven entity and relationship extraction, followed by graph clustering using algorithms like "Leiden Algorithm", and then hierarchical summarisation of communities. At query time, relevant community summaries are retrieved rather than raw chunks.

The Microsoft framework demonstrated that graph structure can meaningfully improve answer quality for questions that require broad synthesis. However, their approach is computationally expensive. The indexing phase involves

many LLM calls to build community summaries, and the resulting system is difficult to update incrementally. It also requires a significant infrastructure investment to run at any reasonable scale.

2.2 fast-graphrag

The fast-graphrag library [1] on GitHub, published by circlemind-ai takes a more pragmatic approach. Rather than building hierarchical community summaries, it focuses on dynamic and incrementally updatable knowledge graphs, and uses PageRank to rank nodes during retrieval. The library is designed to be faster and more lightweight than the Microsoft implementation.

The key contribution of fast-graphrag that informed this project is the use of PageRank scores to score nodes during retrieval. Instead of treating all nodes in a retrieved subgraph as equally relevant, PageRank provides a way to "rank" the nodes that are more central or more transitively connected to the query context.

3 Solution Methodology

The prototype is built as a simple client-server application. The frontend is a static site built with pure html, css, js, and webpack. It is then hosted on GitHub Pages. The backend is a Rust HTTP server that exposes four main endpoints: knowledge graph construction from user-supplied URLs and text, answering queries against a constructed graph, and two auxiliary endpoints for graph and sources retrieval (to show the history).

3.1 Knowledge Graph Extraction

When a user submits URLs or raw text, the backend sends the content to Google Gemini (gemini-3.1-flash-lite) with a structured output prompt that instructs the model to return a list of entities and their directed relationships. The extraction is performed in two passes: a first pass extracts an initial set of entities and relationships, and a second pass sends the same source material back to the model with the existing graph included in the prompt, asking it to find additional relationships that may have been missed. This two-pass approach was an attempt to improve recall without dramatically increasing cost, though it is not a rigorous solution and results vary considerably by content.

3.2 Query Processing

When a user submits a query, the backend first sends the query to Gemini and asks it to identify the key entities mentioned. These become the seed nodes for the personalised PageRank computation. Since entity names as extracted from the query may not exactly match entity names in the graph (e.g., "Marie Curie" vs "mdm. curie"), fuzzy string matching using the Jaro-Winkler algorithm from the strsim Rust crate is used to find the closest matching nodes in the graph.

The core retrieval mechanism is personalised PageRank computed via power method [3].

$$\pi' = \alpha P \pi + (1 - \alpha) v$$

The personalisation vector v assigns a weight of $\frac{1}{|\text{seeds}|}$ to each seed node and zero to all other nodes. The damping factor α is set to 0.85, following the value commonly used in literature. Power method runs for 20 iterations to ensure convergence for small graphs. The result is a score for each node in the graph reflecting the "importance" or "relevance" of the nodes relative to the seed nodes.

The top-10 nodes by PageRank score are then selected and used to construct a subgraph. To ensure coherence, the subgraph includes those top-10 nodes, any edges between them, and any nodes that appear as either source or target in those edges. This subgraph is stored as a list of entity [relationship] entity strings and injected into the final prompt sent to Gemini, which is instructed to answer the user's query using only the provided subgraph. This constraint on the answer source is a deliberate design choice to ensure that the model's response is grounded in the extracted graph and to minimize hallucination.

The full pipeline, from query submission to answer, can be summarised as:

Query → entity extraction → Jaro-Winkler matching → Personalised PageRank → Top-K node selection → Subgraph construction → LLM answer generation

4 Experimental Setup

4.1 Deployment

The backend is a binary that runs in a Docker container on Azure Container Apps. The container image is built on a distroless base to minimise the attack surface and image size. A Cloudflare tunnel sidecar handles the traffic as it was too time consuming to build the networking infrastructure from scratch. Application secrets (API keys and tokens) are stored in Azure Key Vault and injected at runtime.

The frontend is deployed to GitHub Pages. Both the container and the GitHub Pages deployment are automated through GitHub Actions. The container is pushed to Azure Container Registry using Azure Managed Identities and OIDC for keyless authentication.

4.2 Language Of Choice: Rust

Rust was chosen deliberately, partly as a learning exercise and partly because it offered memory safety guarantees without a garbage collector, which felt appropriate for a long-running server process. In practice, the choice made certain things harder than they needed to be. For example, the async ecosystem has a steeper learning curve than Python or Go equivalents, and working with complex nested data structures required careful thinking about ownership and borrowing.

4.3 LLM Selection

Google Gemini (gemini-3.1-flash-lite) was used for all LLM calls, primarily because of its structured output support and because it was available at a cost point suitable for this project.

4.4 Test Corpus

The system was tested with Wikipedia articles on Marie Curie, radioactivity, radium, and the history of radiotherapy, using the query "What discoveries by Marie Curie led to later advances in medical imaging?" as the primary evaluation case.

4.5 Data Persistence.

The knowledge graph is stored entirely in memory within the Rust process. There is no database. This means the graph is lost every time the container restarts, and there is no support for concurrent users building separate graphs.

5 Results and Discussion

The system produced plausible multi-hop answers for the Marie Curie test case. Starting from the seeds "Marie Curie" and "medical imaging" (after fuzzy matching to graph entities), personalised PageRank surfaced nodes including "radioactivity," "radium," "polonium," "radiotherapy," and "X-ray" in the top-10, with lower scores for more distantly connected nodes. The constructed subgraph contained edges like:

- Marie Curie ⟶ discovered ⟶ Radium
- Radium ⟶ used in ⟶ Radiotherapy
- Radiotherapy ⟶ is a form of ⟶ Medical Imaging Treatment

The final answer generated by Gemini, conditioned on this subgraph, correctly traced the chain from Curie's discovery of radium to its use in cancer treatment, which is a reasonable answer to the multi-hop query.

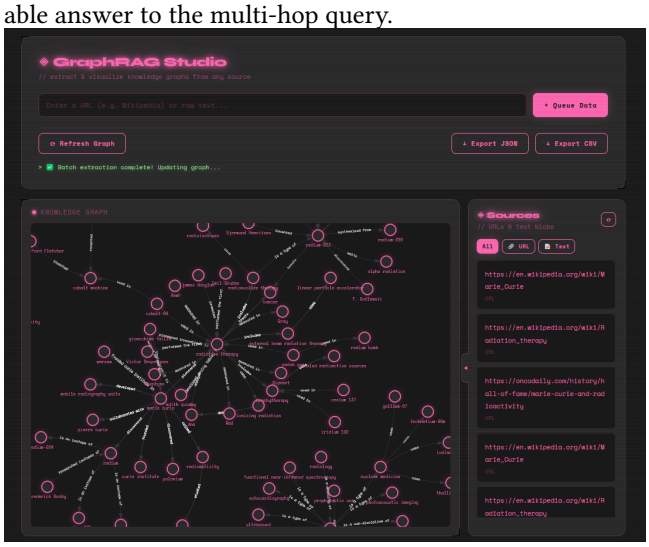

Figure 1: Extracted knowledge graph

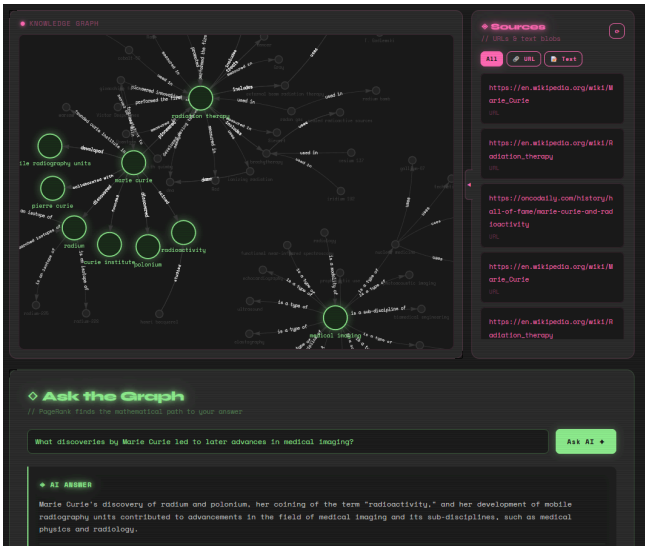

Figure 2: Knowledge graph with top-10 nodes highlighted by personalised pagerank scores

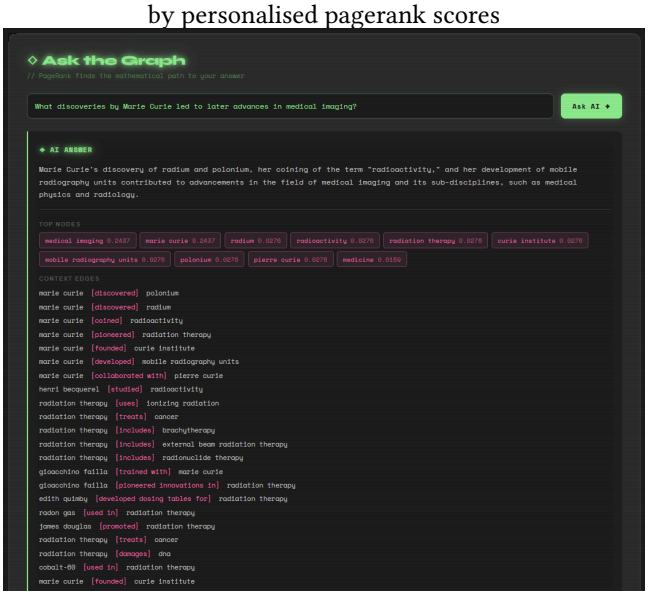

Figure 3: Generated Answer and top-10 nodes with list of context edges

That said, "plausible" is doing a lot of work in that sentence. No rigorous evaluation was conducted: no benchmark dataset, no comparison against a baseline vector-RAG system, and no human evaluation beyond informal inspection. The quality of the graph varied significantly depending on the source material and how cooperative the LLM was with the extraction prompt. On some inputs, the extraction produced well-structured entities and relationships. While on others, it produced overly verbose node names or duplicate entities with slightly different phrasing that the Jaro-Winkler matcher could not reliably deduplicate.

The most interesting observation was qualitative: the graph-based retrieval felt more interpretable than a vector search. The subgraph injected into the final prompt is human-readable. A person can look at the context edges

and understand why those facts were selected. That interpretability seems valuable, even if the accuracy might not be better than the traditional approach.

6 Technical Challenges and Risk Assessment

6.1 Prompt Engineering

Getting the LLM to consistently return an acceptable knowledge graph was harder than expected. Initial prompts produced inconsistent entity naming and occasional hallucinated relationships. Iterating on the extraction prompt took longer than expected, from reading documentations, researching how other frameworks approached the problem to asking several LLMs to craft example prompts. Fortunately, the biggest improvement to this project came from adjusting the prompt to inject a persona (system instructions for gemini) and relevant context edges. For example, including the existing graph in the second pass prompt and asking the model to find additional relationships it may have missed led to a noticeable increase in graph density and answer quality.

6.2 Azure and Cloudflare Infrastructure Deployment using cdk-terrain (cdktn)

Setting up the Azure infrastructure using cdktn (a cloud development kit for terraform/opentofu) was unexpectedly time-consuming. Getting used to navigating the azure, cloudflare, terraform, and cdktn documentations took a while because there were so many moving parts. There were several points where the deployment failed because of misconfigurations and permission issues. In particular, configuring Key Vault secret injection required a few trials and errors as a missing role assignment and circular dependencies produced unhelpful error messages that took time to diagnose.

6.3 Rust's Learning Curve

Choosing Rust for a project that also involved learning GraphRAG, PageRank, and cloud infrastructure simultaneously was probably overambitious. Rust's ownership model is not inherently difficult, but it requires a different mental model than most other languages, and that cognitive overhead compounded with the other unfamiliar concepts in the project. That said, the compiler's error messages were genuinely helpful, they often pointed directly at the problem.

6.4 Risk Assessment

The main risks in the current build are:

- Data loss on restart: All graph data is in-memory. A container restart clears everything, which makes the system unsuitable for any use case where the graph takes time to build.
- No concurrency isolation: Multiple users share the same in-memory graph. Concurrent requests could produce inconsistent results.
- LLM cost unpredictability: Every ingestion and query request makes multiple LLM API calls. There is only a basic rate limiting and no caching or cost cap in the current implementation.
- Entity disambiguation fragility: The Jaro-Winkler fuzzy matching is a heuristic that fails on entities with short names or names that are superficially similar to unrelated entities.
- No evaluation framework: Without a benchmark, it is impossible to know whether changes to the extraction prompt or the PageRank parameters actually improve answer quality.

7 Conclusion

This project produced a working prototype of GraphRAG with personalised PageRank, deployed to a real cloud environment with a reasonable CI/CD pipeline. It is experimental in the fullest sense, and it has not been evaluated rigorously, it cannot handle concurrent users, it loses its data on restart, and it would need significant work before being considered reliable.

The most durable outcome of the project is conceptual. Working through the implementation provided a much more concrete understanding of why graph-based retrieval is structurally different from vector-based retrieval, and what kinds of queries each approach is better suited for.

If this prototype were to be extended, the most impactful improvements would be: replacing in-memory storage with a proper graph database (Neo4j or a similar store), adding an evaluation framework with ground-truth query-answer pairs, and investing more seriously in entity disambiguation, either through better prompt engineering or through a dedicated named entity recognition model. The personalised PageRank implementation itself is functional and would not need to change substantially as the current bottleneck is in the quality of the graph that feeds into it.

References

[1] Circlemind. 2024. Circlemind-ai/fast-graphrag: Rag that intelligently adapts to your use case, data, and queries. Retrieved from https://github.com/circlemind-ai/fast-graphrag

[2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2025. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Retrieved from https://arxiv.org/abs/2404.16130

[3] Frank van Rest. 2009. A mathematical approach to scalable personalized PageRank. Bachelor's thesis. Retrieved from https://math.leidenuniv.nl/scripties/vanRestBach.pdf

[4] Hayden Wolff. 2023. RAG 101: Demystifying Retrieval-Augmented Generation Pipelines. Retrieved from https://developer.nvidia.com/blog/rag-101-demystifying-retrieval-augmented-generation-pipelines/

Appendix

GitHub Repository: https://github.com/vianho/SC4052-A2